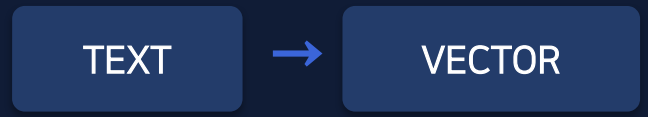
NATURAL LANGUAGE PROCESSING

자연어 처리 (NLP)

텍스트 이해에서 생성 모델까지



KUGGLE 14 기 | 이재헌

NLP 란 무엇인가 ?

인간의 언어를 컴퓨터가 분석 · 이해 · 생성하도록 만드는 AI 기술

입력

"배송이 너무 늦어요"

→

표현

토큰화 → 숫자 벡터

→

모델

패턴 학습 · 문맥 분석

→

출력

부정 감정 / 고객 응대

주요 활용 | 감정 분석 · 문서 분류 · 검색 · 정보 추출 · 번역 · 요약 · 질의응답

핵심 : 자연어는 그대로 계산할 수 없으므로 모델이 처리할 수 있는 표현으로 변환해야 한다 .

NLP 가 어려운 이유 : 같은 단어도 뜻이 달라진다

모호성 · 문맥 의존성

"배가 아프다" → 신체
"배를 먹었다" → 과일
"배를 탔다" → 선박

같은 표면형이라도 주변 단어에 따라 의미가 달라진다 .

언어 변화 · 관계 파악

"이 영화 미쳤다" → 상황에 따라 긍정일 수 있음
"철수가 영희에게 책을 줬다"
→ 누가 / 누구에게 / 무엇을 구분해야 함

단어 빈도만으로는 이러한 관계를 놓치기 쉽다 .

따라서 NLP 는 단어 자체뿐 아니라 순서 , 문장 구조 , 앞뒤 문맥을 함께 다뤄야 한다 .

NLP 가 중요한 이유 : 비정형 텍스트를 데이터로

정보 처리

뉴스 · 리뷰 · 상담 기록을 자동 분류

수작업으로 처리하기 어려운 양을 분석

정보 검색

질문과 관련된 문서 탐색

문서에서 인물 · 날짜 · 금액 추출

서비스 자동화

고객 문의 의도 파악

챗봇 · 번역 · 문서 요약

생성형 AI

텍스트의 패턴과 문맥 학습

질의응답 · 초안 작성 · 코드 생성

데이터사이언스 관점 : 문장을 분석 가능한 변수 · 특징으로 바꾸면 분류 , 예측 , 탐색이 가능해진다 .

NLP 활용 사례 : 입력과 출력으로 이해하기

감정 분석

입력 “화질은 좋지만 배터리가 짧다”

출력 긍정 요소 : 화질 / 부정 요소 : 배터리

정보 추출

입력 “ 11 월 20 일 서울에서 세미나 개최”

출력 날짜 : 11 월 20 일 / 장소 : 서울

요약 · 번역

입력 긴 회의록 또는 외국어 문서

출력 핵심 문장 요약 / 다른 언어로 변환

서로 다른 과제라도 공통적으로 텍스트를 모델 입력으로 변환하는 단계가 필요하다 .

전처리 : 원문을 모델이 다룰 수 있게 정리

예시 : “ 배송이 정말 늦네요 !!! ㅠㅠ”

원문 수집

리뷰 · 댓글 · 문서 확보

→

정규화 · 정제

중복 공백 · 불필요한 태그 처리

→

토큰화

[배송 , 이 , 정말 , 늦 , 네요 , !]

→

모델 입력

토큰 ID / 벡터 변환

주의 : “ 안 좋다” 에서 ‘안’ 을 제거하면 의미가 반대로 바뀔 수 있다 .

전처리는 무조건 많이 지우는 작업이 아니라 분석 목적에 필요한 정보를 보존하는 과정이다 .

토큰화와 불용어 제거 : 한국어는 왜 까다로울까 ?

토큰화 : 문장을 작은 단위로 분해

입력 : " 학생들이 책을 읽었다"

공백 기준 : [학생들이 , 책을 , 읽었다]

형태소 관점 (개념 예시):

[학생 , 들 , 이 , 책 , 을 , 읽 , 었 , 다]

조사 · 어미 때문에 단순 공백 분리로는 부족할 수 있다 .

불용어 제거 : 과제에 따라 선택

분류에서 영향이 작은 고빈도 단어는 제거 후보

하지만 "안" , " 못" , " 별로" 는 감정 · 부정 판단에 중요

고전적 모델은 정제가 중요할 수 있고 , 사전학습 언어 모델은 전용 토크나이저 규칙을 따르는 것이 일반적이다 .

토큰화 단위는 단어 · 형태소 · 서브워드 등으로 달라지며 , 모델별로 적합한 방식이 다르다 .

텍스트 수치화 : 문장을 벡터로 바꾸는 세 가지 관점

BoW

단어별 등장 횟수

문서마다 길이가 같은 희소 벡터

TF-IDF

등장 빈도 × 문서 간 희귀도

문서에서 특징적인 단어 강조

임베딩

단어를 학습된 밀집 벡터로 표현

의미적 유사성을 반영할 수 있음

비교 기준 : 무엇을 세는가 ? / 단어 간 관계를 학습하는가 ? / 문맥에 따라 표현이 달라지는가 ?

BoW·TF-IDF 는 문서별 빈도 기반 표현 , Word2Vec 은 단어별 학습된 벡터 표현이다 .

문장 벡터화 실습: 숫자로 바꾸면 무엇을 비교할까?

문서 A “고양이 귀엽다”

문서 B “강아지 귀엽다”

	고양이	강아지	귀엽다
A	1	0	1
B	0	1	1

BoW 벡터

A = (1, 0, 1) B = (0, 1, 1)

각 열은 동일한 단어를 의미

코사인 유사도

(A·B) / (||A|| ||B||) = 1/2

공통 단어 “귀엽다”가 유사도에 기여

주의 | 단어 빈도만 비교하므로 “좋다”와 “안 좋다”의 의미 차이를 놓칠 수 있다.

핵심: 같은 어휘 공간에서 벡터를 만들면 문서 간 유사도를 계산할 수 있다.

BoW 와 TF-IDF: 같은 단어라도 중요도가 다르다

문서 1: " 고양이 고양이 귀엽다" **문서 2: " 강아지 귀엽다"**

문서	고양이	강아지	귀엽다
D1	2	0	1
D2	0	1	1

TF-IDF 의 직관

TF: 한 문서에서 자주 등장할수록 ↑

IDF: 여러 문서에 흔할수록 ↓

→ "귀엽다" 보다 문서별 고유 단어가 두드러짐

IDF **예시 :** log(N / df) | N = **전체 문서 수** , df = **해당 단어가 등장한 문서 수**

실제 구현에서는 0 방지 · 평활화 등으로 IDF 수식이 조금씩 달라질 수 있다 .

Word2Vec: 주변 단어를 예측하며 의미를 학습

예시 문장 : “ 나는 오늘 따뜻한 커피를 마셨다”

CBOW: 주변 → 중심 단어

[오늘] [따뜻한] [?] [마셨다]
주변 단어를 입력으로 받아 중심 단어 “커피를” 예측
→ 주변 문맥을 압축해 중심 단어 학습

Skip-gram: 중심 → 주변 단어

[커피를] → [따뜻한], [마셨다] ...
중심 단어로 주변 단어를 예측
→ 함께 나타나는 단어 관계 학습

학습 후 : 비슷한 맥락의 단어 벡터는 공간에서 가까워질 수 있다 . 단 , 단어마다 보통 하나의 고정 벡터 .

한계 : “ 은행” 이 금융기관인지 강둑인지에 따라 벡터가 달라지지 않는 정적 임베딩이다 .

정적 임베딩의 한계: 문맥에 따라 달라지는 의미

문장 1 | 은행에 돈을 맡겼다

은행 = 금융기관

주변 단어: 돈, 맡기다

문장 2 | 강가의 은행에 앉았다

은행 = 강둑(문맥상)

주변 단어: 강가, 앉다

Word2Vec | 정적 임베딩

“은행” → 동일한 단어 벡터

문맥별 뜻을 직접 구분하기 어려움

Transformer | 문맥 임베딩

“은행” → 문장마다 다른 표현

주변 토큰과의 관계를 함께 반영

발전 흐름 | 단어의 유사성 학습 → 문장 내 관계 학습 → 문맥에 맞는 표현 생성

핵심: Word2Vec은 단어별 고정 벡터, Transformer는 문장별 문맥 표현을 학습한다.

전통 ML → RNN → LSTM: 단어 순서를 학습하다

전통적 머신러닝

BoW/TF-IDF → 분류기

예 : 로지스틱 회귀 ·SVM

장점 : 간단하고 해석이 쉬움

한계 : 단어 순서 · 문맥 표현 제한

RNN: 순차적 상태 전달

"나는" → "오늘" → "간다"

$h_1 \rightarrow h_2 \rightarrow h_3$

이전 정보를 은닉 상태에 전달

한계 : 긴 거리 정보 학습이 어려움

LSTM / GRU: 게이트

필요한 정보는 유지하고

불필요한 정보는 줄이는 구조

장기 의존성 문제 완화

단 , 순차 계산의 제약은 남음

예 : 긴 문장의 앞부분 주어가 뒷부분 동사 해석에 영향을 주는 경우 , 먼 정보 연결이 중요하다 .

Attention 과 Transformer: 문장 전체의 관계를 직접 본다

“철수는 우산을 챙겼다 . 그는 비가 올 것을 알았다 .”

철수 | 우산 | 그는 | 비 | 알았다

“그는” 이 누구를 가리키는지 판단할 때 “철수” 에 높은 주의를 둘 수 있다 .

Self-Attention

Q(질문) 와 K(관련성) 를 비교해 가중치 계산 → V(정보) 를 가중합

Transformer

Self-Attention + FFN 을 중심으로 문맥을 처리 ; 학습 시 토큰 병렬 계산 가능

핵심 수식 : $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$ | 그림은 개념적 예시다 .

Self-Attention: 관련 단어에 얼마나 주목할까?

"철수는 우산을 챙겼다. 그는 비가 올 것을 알았다."

① Query
"그는"의 의미를 파악

→

② Key와 비교
각 단어와 관련성 점수 계산

→

③ Value 가중합
관련 정보가 문맥 표현에 반영

Q·K 유사도 → softmax → 가중치 → V의 가중합

"그는"이 참조하는 정보의 예시 가중치



핵심 | Attention은 단어 간 관련성에 따라 정보를 선택적으로 결합한다.

아래 수치는 작동 원리를 보여주기 위한 가상 예시이며 실제 모델 출력이 아니다.

GPT 와 BERT: 같은 Transformer, 다른 학습 목표

GPT: 다음 토큰 예측 · 생성

입력 : " 오늘 날씨가"
예측 : " 좋다" → 다음 토큰 반복 생성

주로 Decoder-only 계열
Causal Mask 로 미래 토큰 참조 제한
문장 생성 · 대화 · 코드 작성에 활용

BERT: 양방향 문맥 표현

입력 : " 오늘 [MASK] 가 좋다"
예측 : "[MASK] = 날씨"

Encoder-only 계열
양쪽 문맥을 활용한 표현 학습
분류 · 개체명 인식 등에 활용

정리 | 텍스트 → 토큰 → 벡터 → 문맥 모델 → 분류 · 검색 · 생성

참고 : Vaswani et al.(2017), Mikolov et al.(2013), Devlin et al.(2019), Radford et al.(2018).